CLASSROOM PRESENTATION

Logistic Regression

From a linear score to a classification decision

Intuition • Sigmoid • Log Loss • Model Training • Evaluation • GridSearchCV

NO DERIVATIVES

STEP-BY-STEP EXAMPLES

What students will be able to do

By the end, they should understand the model—not just run the code.

Explain the need

Recognize binary classification and why an ordinary regression output is not a probability.

Calculate a prediction

Work from z = mx + c to Sigmoid probability and then the class label.

Explain model learning

Interpret Log Loss and describe how coefficients are adjusted to reduce it.

Build and assess

Train, evaluate, cross-validate and tune Logistic Regression in Scikit-learn.

12 topics • one connected story

Organized into four blocks: Understand → Predict → Learn → Apply

01–02

Why classification?

Pass/Fail data
Linear Regression limitation

03–06

How prediction works

Linear score → Sigmoid
Threshold → Decision boundary

07–08

How it learns

Log Loss
Gradient Descent intuition

09–12

How we use it

Multiple features → Evaluation
Python → CV + GridSearchCV

One dataset and one set of symbols are reused through the foundational explanation.

Start with a familiar problem

Can study hours help predict whether a student passes?

Study hours (X)	Result (y)
1	0 · Fail
2	0 · Fail
3	0 · Fail
4	1 · Pass
5	1 · Pass

OUR QUESTION

What happens when a new student studies 6 hours?

0 = FAIL

1 = PASS

This small dataset is illustrative; real training uses more observations and validation data.

Regression vs classification

Same input variable; a different type of output.

LINEAR REGRESSION

Question: How many marks will the student score?

Output: a continuous number, such as 72.5

LOGISTIC REGRESSION

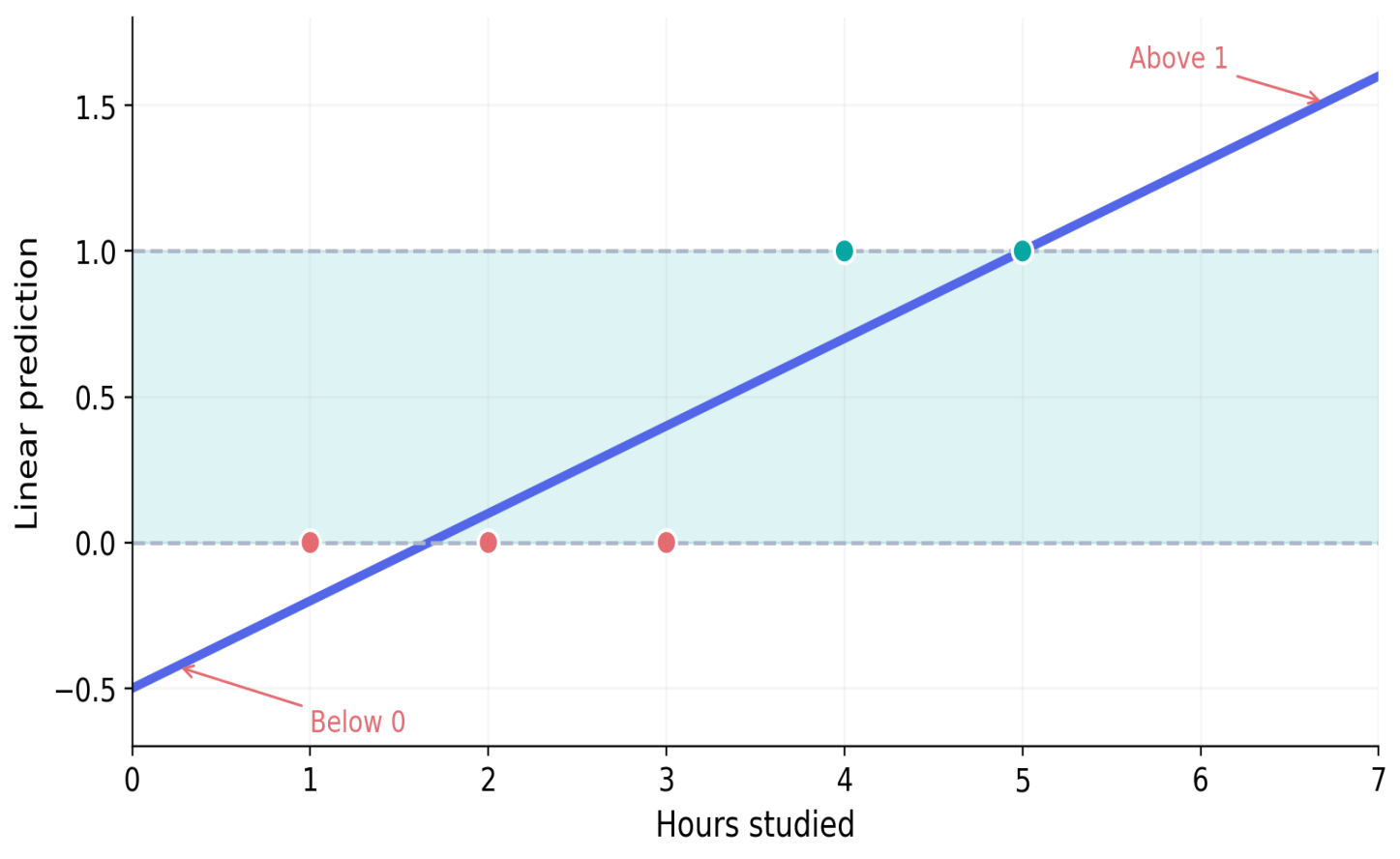
Question: Will the student pass?

Output: a probability, followed by Pass or Fail

Logistic Regression is a classification algorithm despite the word “regression” in its name.

Why not predict binary outcomes with a line?

An ordinary linear prediction can be smaller than 0 or larger than 1.



ILLUSTRATIVE LINE

$$\hat{y} = 0.3x - 0.5$$

$x = 1 \rightarrow -0.2$

$x = 6 \rightarrow 1.3$

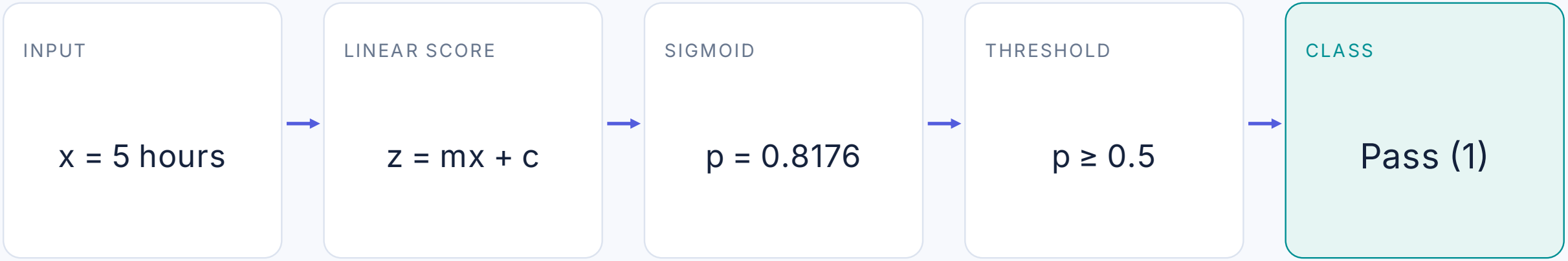
A probability must stay within 0 and 1.

Ordinary Linear Regression can be thresholded, but it is not designed to model valid binary probabilities.

PREDICTION

The full prediction idea—in one view

Logistic Regression first predicts a probability, then a class.



Probability ≠ predicted class

Step 3 — Start with z = mx + c

The linear equation produces an intermediate score, not a probability.

WORKED EXAMPLE

$$z = mx + c$$

$$m = 1.5 \quad c = -6 \quad x = 5$$

$$z = (1.5 \times 5) - 6 = 1.5$$

Remember

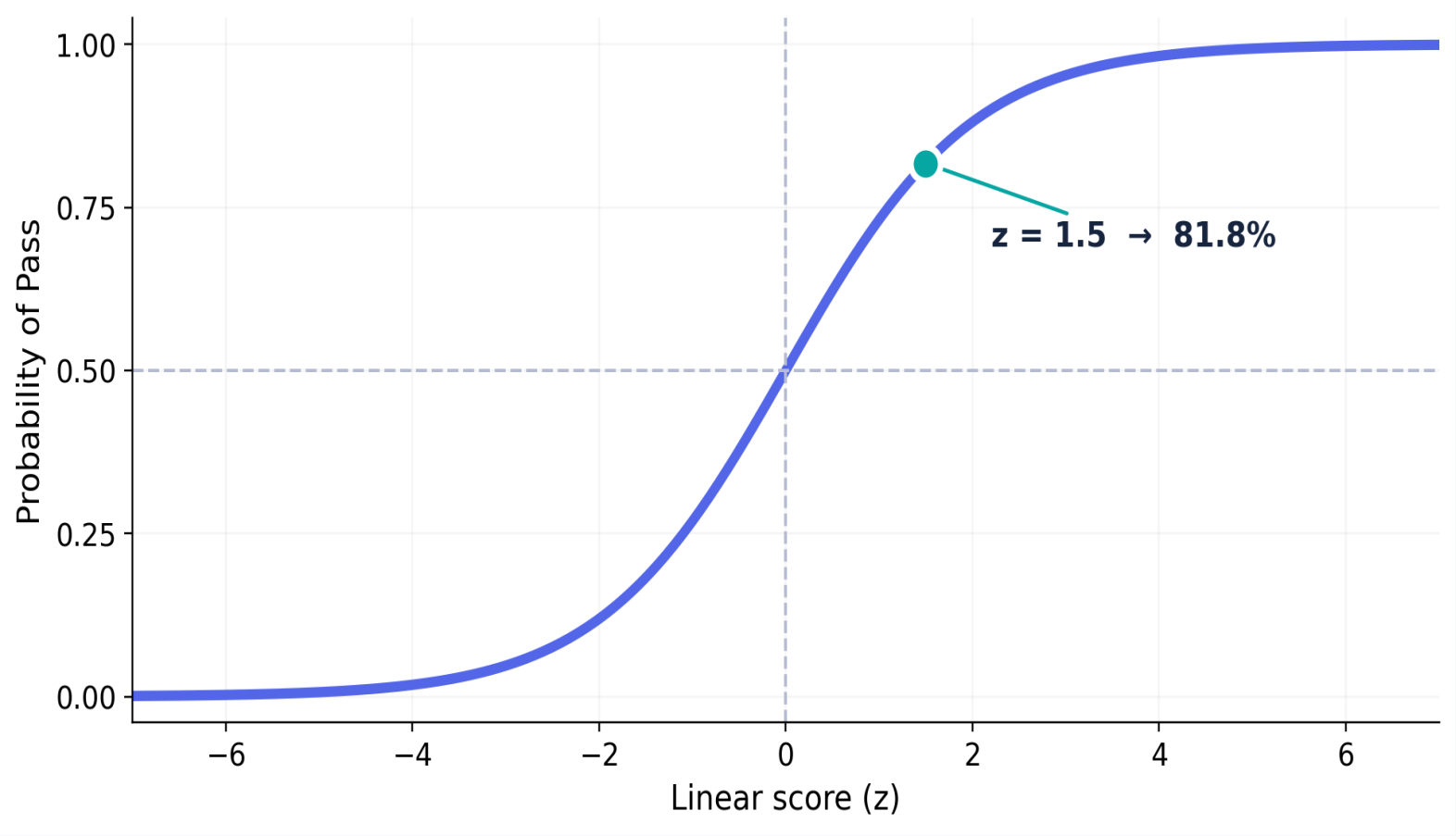
$$z = 1.5$$

The score can be any real number. We still need to turn it into a valid probability.

m = 1.5 and c = −6 are assumed for this hand calculation; they are not fitted values from the small table.

Step 4 — Sigmoid makes the score a probability

It maps any value of z to a number between 0 and 1.



SIGMOID FORMULA

$$p = 1 / (1 + e^{-z})$$

z < 0 → p < 0.5

z = 0 → p = 0.5

z > 0 → p > 0.5

Here e is a mathematical constant, approximately 2.718.

Calculate one probability by hand

For the student who studied five hours, z = 1.5.

01 Linear score

$z = 1.5$

02 Apply Sigmoid

$p = 1 / (1 + e^{-1.5})$

03 Evaluate exponent

$e^{-1.5} \approx 0.2231$

04 Final probability

$p \approx 0.8176 = 81.76\%$

Interpretation: the model estimates an 81.76% probability of the student passing.

The same model predicts for every student

Keep m = 1.5 and c = −6 fixed; change only the input x.

Hours (x)	Score z	P(Pass)	Class at 0.5
1	−4.5	0.011	Fail
2	−3.0	0.047	Fail
3	−1.5	0.182	Fail
4	0	0.500	Pass
5	1.5	0.818	Pass
6	3.0	0.953	Pass

QUESTION: why does 4 hours become Pass?

Step 5 — Threshold converts probability to a class

A 0.5 threshold is common, but it can be changed.

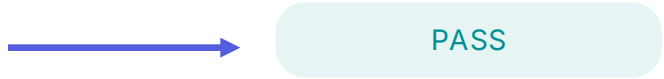
RULE

$p \geq 0.50 \rightarrow$ Pass (1)

$p < 0.50 \rightarrow$ Fail (0)

OUR EXAMPLE

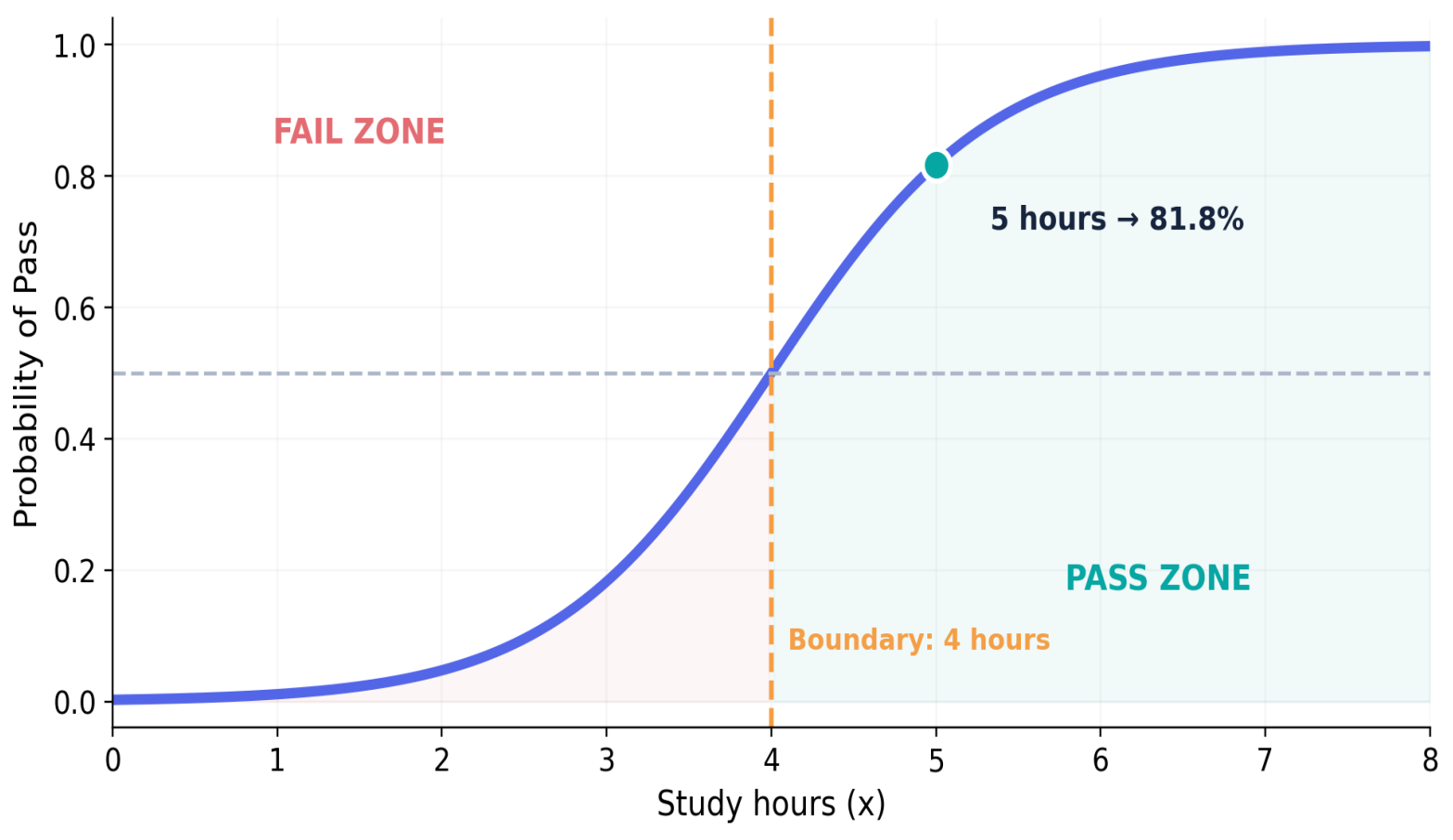
$p = 0.8176$



Thresholding happens after the probability is calculated—not during the Log Loss calculation.

Step 6 — The decision boundary

With a 0.5 threshold, the class changes where Sigmoid probability is 0.5.



CALCULATE IT

$$p = 0.5$$

$$z = 0$$

$$1.5x - 6 = 0$$

$$x = 4 \text{ hours}$$

Decision boundary is a point with one feature; a line in the original feature space with two features.

Quick check — predict before revealing

Use the illustrative model $z = 1.5x - 6$.

A student studied 3 hours.

1. What is z?

2. Is p below or above 0.5?

3. Pass or Fail?

Quick check

Use the illustrative model z = 1.5x − 6.

A student studied 3 hours.

1. What is z?

2. Is p below or above 0.5?

3. Pass or Fail?

TRAINER ANSWER

$$z = -1.5$$

$$p \approx 0.182$$

FAIL (0)

Step 7 — Where do m and c come from?

Training searches for coefficients that produce good probabilities.

BEFORE TRAINING

m and c are not known yet

We provide hours studied and the actual Pass/Fail labels.

AFTER TRAINING

m and c have been learned

The model uses those learned values to predict probabilities for new students.

The earlier coefficients 1.5 and −6 were illustrative—they were not outputs of fitting this five-row dataset.

Step 8 — Log Loss measures prediction quality

We reward high probability for what happened and penalize high confidence when wrong.

Actual Result	Predicted P(Pass)	Log Loss	Interpretation
Pass (1)	0.90	0.105	Good probability
Pass (1)	0.10	2.303	Confidently wrong
Fail (0)	0.10	0.105	Good probability
Fail (0)	0.90	2.303	Confidently wrong

Lower Log Loss is better. A confidently incorrect prediction receives a large penalty.

Log Loss formula: just two simple cases

The actual label determines which probability matters.

ACTUAL = PASS (y = 1)

$$\text{Loss} = -\ln(p)$$

p = 0.9 → Loss = 0.105

ACTUAL = FAIL (y = 0)

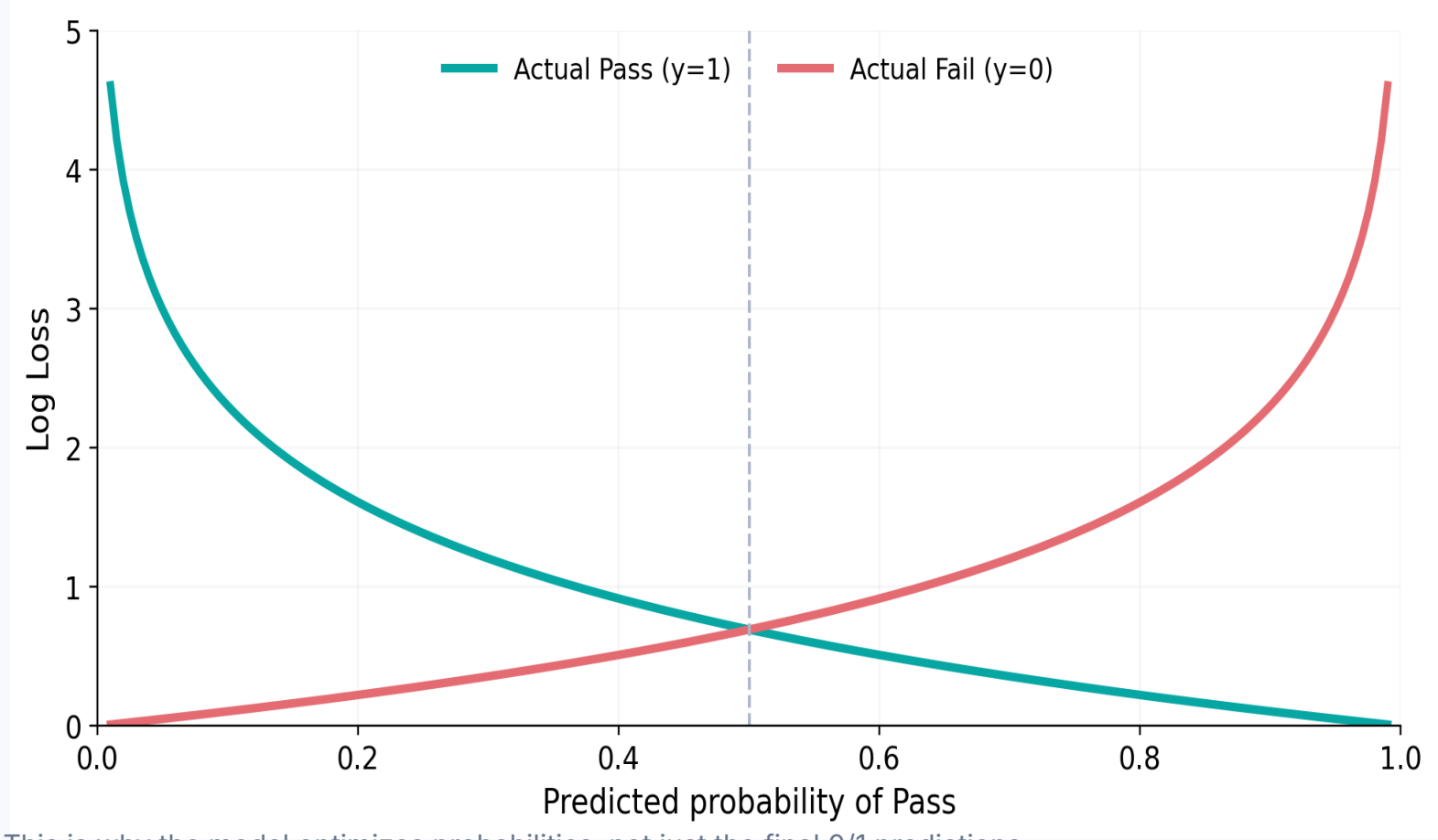
$$\text{Loss} = -\ln(1 - p)$$

p = 0.9 → Loss = 2.303

p always means the predicted probability of Pass (class 1); ln is the natural logarithm.

Visualize what Log Loss penalizes

As a probability becomes confidently wrong, the loss rises sharply.



This is why the model optimizes probabilities, not just the final 0/1 predictions.

Key idea

If the student passes, assigning p = 0.99 is excellent.

If the student passes, assigning p = 0.01 is very costly.

Calculate average Log Loss for three students

The training objective averages loss across observations.

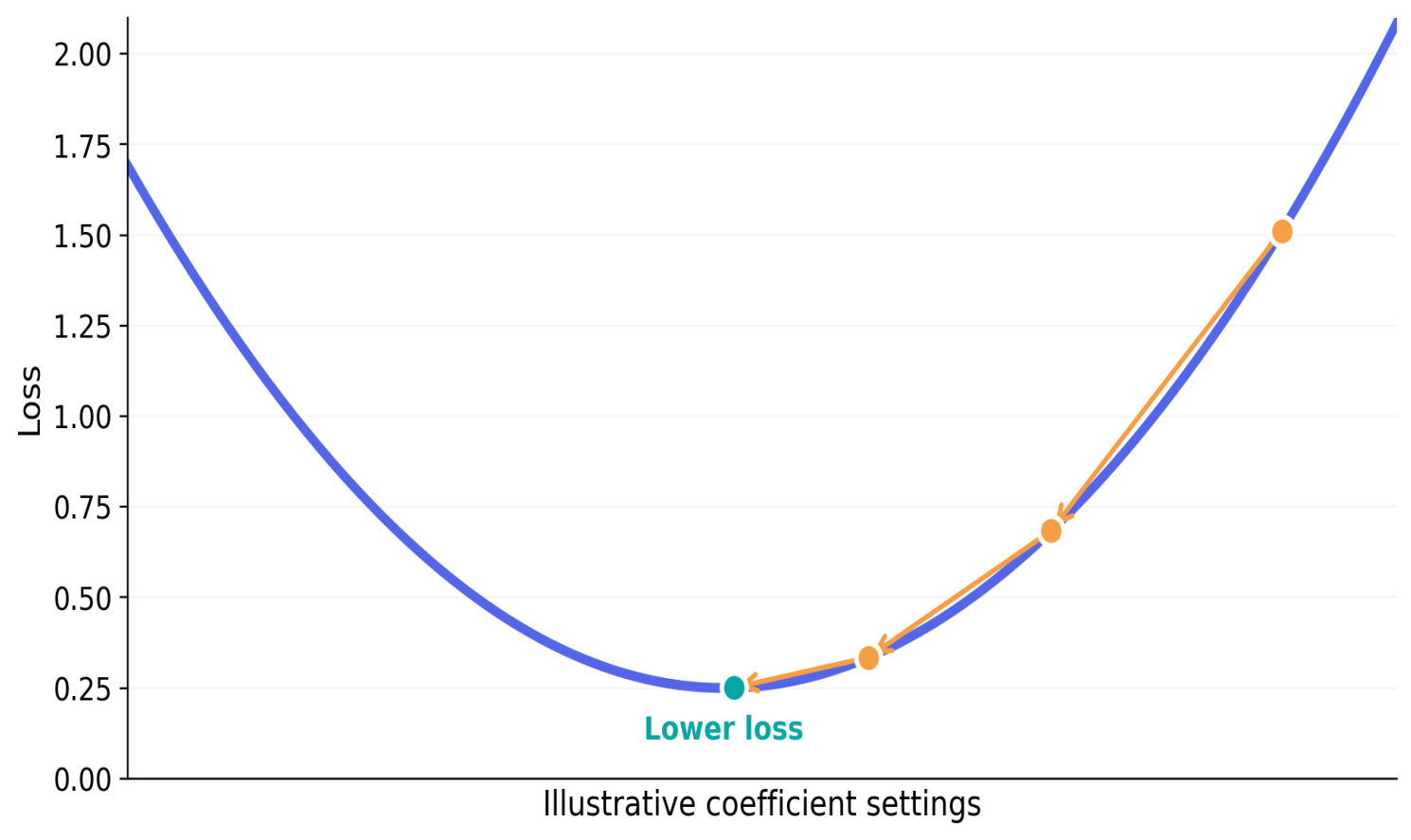
Actual y	Predicted p	Probability of actual label	Individual loss
1 — Pass	0.8	0.8	0.223
0 — Fail	0.3	0.7	0.357
1 — Pass	0.6	0.6	0.511

$$\text{Average Log Loss} = (0.223 + 0.357 + 0.511) / 3 \approx 0.364$$

Goal of model training: find coefficients that make this average loss as small as possible.

Step 8 — Gradient Descent: the intuition

Think of training as moving downhill toward lower loss.



TRAINING LOOP

1. Choose starting m and c
2. Calculate probabilities
3. Measure Log Loss
4. Adjust m and c
5. Repeat until training stops

The downhill diagram illustrates the idea; actual optimization can involve many coefficients and a more complex surface.

Training and prediction are different operations

Training learns the parameters; prediction reuses them.

TRAINING • fit(X, y)

Uses features AND known results.

Calculates probabilities, evaluates loss, and learns coefficients.

PREDICTION • predict(X)

Uses new feature values.

Calculates score → probability → final class using learned coefficients.

We do not know the true label of a new case at prediction time.

Step 9 — Logistic Regression with two features

Extend the linear score to include study hours and attendance.

TWO INPUTS

$$z = w_1x_1 + w_2x_2 + c$$

x_1 = Hours studied

x_2 = Attendance (%)

WORKED EXAMPLE

$z = 0.8x_1 + 0.05x_2 - 7$

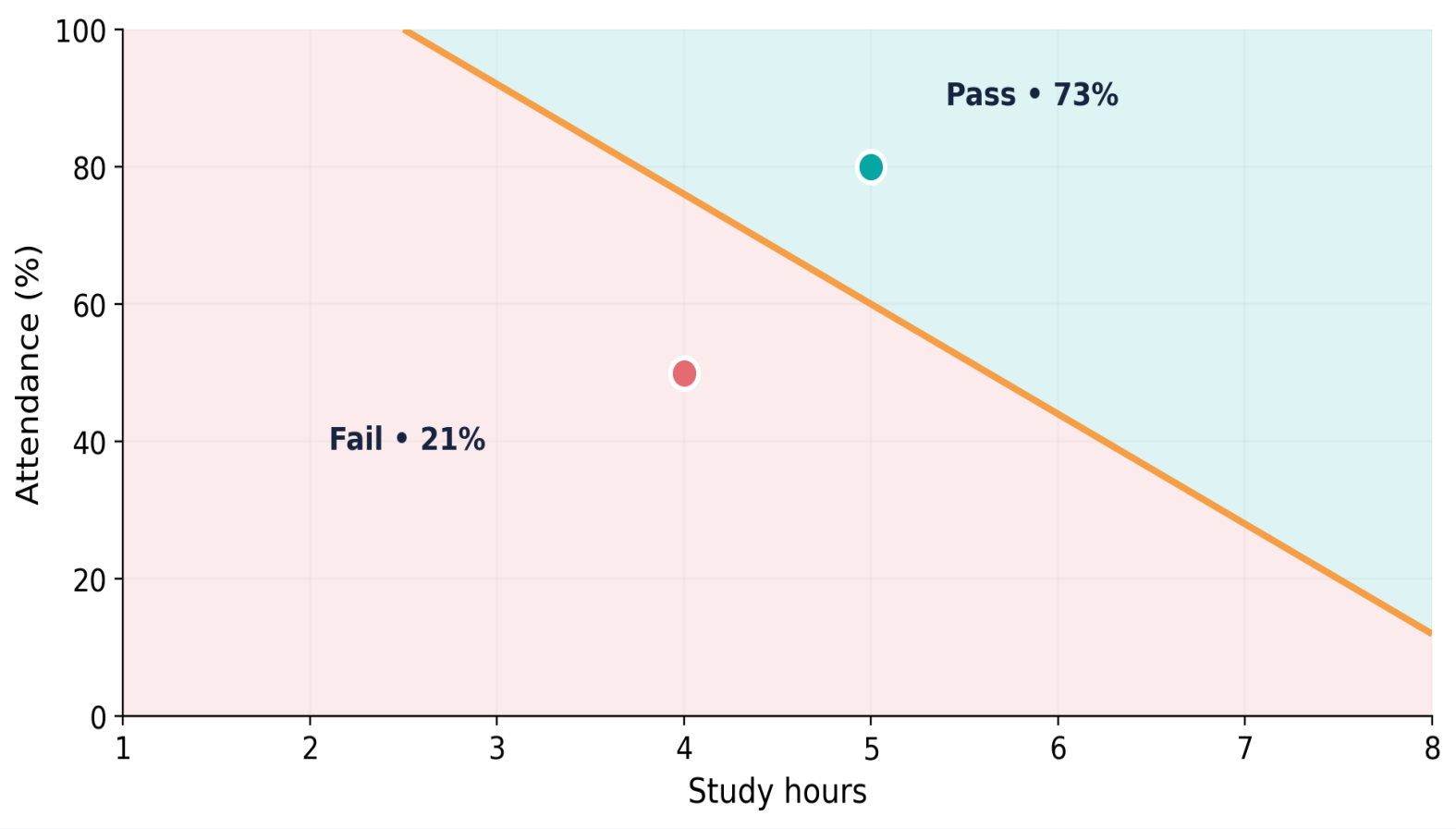
$x_1 = 5, \quad x_2 = 80$

$z = 1 \rightarrow p \approx 0.731$

Coefficients above are illustrative. With multiple features, each feature contributes to the score.

With two features, the boundary becomes a line

A student can cross the decision boundary by changing either feature.



Two student points are illustrative: (5 hours, 80%) ≈ 73% Pass; (4 hours, 50%) ≈ 21% Pass.

SAME RULE

z = 0 means
p = 0.5

The combination of hours and attendance decides which side of the boundary the student falls on.

What do the coefficients tell us?

The coefficient sign shows the direction of a feature's effect on the model score.

Positive coefficient

As this feature increases (others held fixed), the model score increases, so P(class 1) increases.

Negative coefficient

As this feature increases (others held fixed), the model score decreases, so P(class 1) decreases.

Coefficient near zero

A one-unit change in this feature has little effect on the model score, all else fixed.

Coefficients are not "percentage-point probability changes." They act on the score before Sigmoid.

Step 10 — Evaluate classification predictions

A confusion matrix separates correct decisions from the two error types.

	PREDICTED CLASS: Fail (0)	PREDICTED CLASS: Pass (1)
Actual Fail (0)	TN = 16 Correct Fail	FP = 4 Wrong Pass
Actual Pass (1)	FN = 3 Wrong Fail	TP = 17 Correct Pass

ILLUSTRATIVE

40 students

33 correct
7 incorrect

Definitions use Pass as the positive class.

Five metrics students should recognize

Each metric answers a different question.

Metric	Formula	What it asks
Accuracy	(TP + TN) / All	How often is the prediction correct?
Precision	TP / (TP + FP)	When we predict Pass, how often correct?
Recall	TP / (TP + FN)	Of actual Pass cases, how many found?
F1 Score	2 × (P × R) / (P + R)	How balanced are Precision and Recall?
Log Loss	Average probability penalty	How good are the probabilities?

Use ROC-AUC later when introducing ranking performance across decision thresholds.

Step 11 — Prepare the practical workflow

Use a standard Scikit-learn dataset and reserve test data.

IRIS: VERSICOLOR (0) vs VIRGINICA (1)

01	Load dataset	100 flowers • 2 classes
02	Pick features	Petal length + petal width
03	Train/test split	75 train • 25 test
04	Scale + fit	Pipeline handles both
05	Predict + evaluate	Classes, probabilities, metrics

Fit preprocessing on training data—not on the entire dataset before splitting.

PRACTICAL

Python 1/3 — Load data and split

Binary Iris classification using two features.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()
mask = iris.target != 0
X = iris.data[mask][:, [2, 3]]
y = (iris.target[mask] == 2).astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
```

DATA

100 samples

2 classes

2 features

stratify keeps class proportions similar in each split.

Python 2/3 — Train and predict

Scikit-learn learns coefficients when we call fit().

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

model = Pipeline([
    ("scaler", StandardScaler()),
    ("logreg", LogisticRegression(max_iter=1000))
])

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]
```

THE BIG THREE

fit()

Learn from training rows.

predict()

Return class labels.

predict_proba()

Return class probabilities.

PRACTICAL

Python 3/3 — Evaluate the fitted model

These are actual results from the reproducible Iris example.

```
from sklearn.metrics import (
    accuracy_score, confusion_matrix,
    classification_report
)

print(accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

ACTUAL TEST RESULTS

84%

Accuracy (21 / 25 correct)

Precision: 90%

Recall: 75%

Actual held-out confusion matrix: TN=12, FP=1, FN=3, TP=9 (test split uses random_state=42).

PRACTICAL

Step 12 — Cross-validation checks stability

Instead of trusting one split, repeat validation across several folds.



Average the 5 validation scores → cross-validation score

When using a Pipeline, scaling is fitted separately within each training fold, preventing leakage.

GridSearchCV: compare different model settings

We try multiple C values and select using cross-validation.

C value	Mean CV accuracy	Interpretation
0.01	94.67%	Stronger regularization
0.1	94.67%	
1	96.00%	Highest score (tie)
10	96.00%	Highest score (tie)
100	96.00%	Highest score (tie)

Best C = 1 (first tied candidate in this grid) • Best mean CV accuracy = 96.00%

C controls regularization strength: smaller C means stronger regularization. Results use the 75 training rows.

PRACTICAL

GridSearchCV: the complete tuning pattern

Pipeline + StratifiedKFold + GridSearchCV

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
param_grid = {"logreg__C": [0.01, 0.1, 1, 10, 100]}

grid = GridSearchCV(
    model, param_grid=param_grid, cv=cv, scoring="accuracy"
)
grid.fit(X_train, y_train)

print("Best parameters:", grid.best_params_)
print("Best CV score:", grid.best_score_)
print("Test accuracy:", grid.score(X_test, y_test))
```

The double underscore targets a parameter inside the pipeline step named “logreg”.

RECAP

The whole algorithm in four ideas

A clear mental model matters more than memorizing implementation details.

LINEAR SCORE

$z = mx + c$

Combine the input features.

PROBABILITY

$p = 1 / (1 + e^{-z})$

Sigmoid keeps output in 0–1.

LEARNING

Minimize Log Loss

Find useful m and c from data.

CLASSIFICATION

$p \geq \text{threshold} \rightarrow 1$

Convert probability into a label.

fit() learns • predict_proba() estimates • predict() classifies

THE ONE-SENTENCE TAKEAWAY

Logistic Regression uses a linear score to estimate a probability, learns from Log Loss, and turns the probability into a class.

Next: run the Iris example, inspect probabilities, and tune C with cross-validation.